Hierarchical Multi-Label Text Classification

YEJUN OH, Department of Data Science, Korea University, Republic of Korea

Additional Key Words and Phrases: LabelGCN, mxbai-embed-large-v1, Pseudo-labeling, Multi-attention head, self-training

1 Introduction

The demanded task is to perform product review classification without using any labeled data for reviews about Amazon products. We have natural language corpus data for training and testing, and class data for hierarchy and elaborating the related keywords.

First of all, I used pre-trained embedding model for mapping the courpus and labels on the same vector space to calculate the score about similarity among them. Then I recalibrated the score by using class-keyword matching and hierarchy among the labels to render more trustful silver labels.

LabelGCN model is composed of an adjacency matrix, stacked GCN layers with multi-head for rich and precise calculation , and skip-connection. The model is then trained with the silver labels with AdamW optimizer and another scheduler.

Test data are fed to the model to inference the final label for test set. In accordance with the instruction for submission, the model forces to return at least two and at most three labels for each product review. The final performance(F1 score) result was 0.38187.

2 Implementation

You can check the code and the result with some .pt files to easily reproduce the result on the github repository. Below are the steps for implementing the final LabelGCN model.

2.1 Silver label generation

We have three text files to learn so that I defined three functions for each of them. *load_classes* function matches each ID for category label to name of it. It connects these like a bidirectional dictionary that is available for searching for the name with the ID, and vise versa. It became the very axis of the entire code. *load_keywords* function maps each ID to keywords related to each label. At the beginning the keywords are stored with the name of each category, not of ID, therefore it is very useful to match with the ID to use index for convenient search. *load_hierarchy* function helps to connect each label with parent-child relationship for the latter GCN model. It mapped the relationship with bidirectional way to find each other more quickly. Also we should start indexes from 0 so that the model can calculate them all with appropriate way.

We should also load the corpus for train and test data sets for later training and inference process. The function *load_corpus* opens each file and preprocesses them into "pids" and "texts" so that we can adjust the embedding technique to only the text part. The function can take both train and test data and process them into each variable. We can see that there are about thirty thousands train docs and twenty thousands test docs.

Next I connected each label and keyword data with colon into a string to make labels into real meaningful document data. For example you can see the first result like this.

- `grocery_gourmet_food`: snacks, condiments, beverages, specialty_foods, spices, cooking_oils, baking_ingredients, gourmet_chocolates, artisanal_cheeses, organic_foods

Author's Contact Information: Yejun Oh, oyjgod02@korea.ac.kr, Department of Data Science, Korea University, Seongbuk-gu, Seoul, Republic of Korea.

2025. ACM XXXX-XXXX/2025/12-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

To embed the documents to numeric vectors we should choose one of pre-trained embedding model to make them into vector space. At first I chose *TfidfVectorizer* from scikitlearn library. But I noticed that was too naive for the embedding so I'd moved into *BAAI/bge-small-en-v1.5* and experienced the performance has soared. My final choice was *mixedbread-ai/mxbai-embed-large-v1* from "Mixedbread AI " which is frequently used for RAG. The tokenizer and the pre-trained model were loaded first. Then the tokenizer cut each document into tokens of manageable length, which I set to 512. Then normalize it to be easy in calculating cosine similarity when the embeddings come out from the *mxbai* model. Then later we can use them with simple inner product. At this point you can jump this and next code if you have the "full_processed_data.pt" file saved for skip this step. Loading *mxbai* model is time-consuming, and it took at least 30 minutes to fully load. Therefore I've uploaded the link for that .pt file because of the size. Please check this out to reproduce the result.

Finally we can render the pseudo-labels for each document. We have L2-normalized embedding, one is for class("class_embs") and the other is for document("train_embs"). First adjust dot product (i.e. inner product) between them for checking how much each document resembles with labels. And here are my readjustment procedure to strengthen the silver labels. We can see that there can be some keywords which exist both inside of the document and the label-keyword set. If the keyword related to the silver label is also included in the document, we additionally give some positive bonus point to that silver label (in code, alpha is 0.4).

$$S_{i,j}^{(1)} = S_{i,j}^{(0)} + \alpha \cdot \mathbf{1}[\text{doc } i \text{ contains keyword of label } j]$$

Also we can use the relationship between the parents and the children. We can assume that if the child label is weighty, probably so do the parent. But it seems that usually the child label might be relatively stronger than the parent one due to some reason. So I readjusted the parent's core by using the child's one as the lower bound. But for not to overwrite them all with the child's, I set the ratio to weaken that propagation as 0.5 (which is the value of beta below).

$$S_{i,j}^{(2)} = \max(S_{i,p}^{(1)}, \beta \cdot S_{i,c}^{(1)})$$

Additionally I put some power function for flatten the score especially to oppress the higher-scored label (in code, gamma is 0.8).

$$S^{(3)} = (S^{(2)})^{\gamma}$$

There is still one more step left. We can normalize the distribution for each document by scaling the score to divide each one by maximum score per document. But for preventing the zero-divided situation, don't forget to make sure there is no zero denominator.

$$S_{i,:}^{(4)} = \frac{S_{i,:}^{(3)}}{\max_j S_{i,j}^{(3)} + \epsilon}$$

By this four steps we can get reasonable score distribution among the pseudo-labels. Remained step is just using top 20 labels and store them into PyTorch file(.pt).

2.2 Training process

Now the silver-labeling step is over, therefore move into the LabelGCN model. First we should make an adjacency matrix as we always did in the class time. Assign the parent-child relationship arc with one. And adjust self-loop by adding identity matrix for stabilizing each node from swing too much within adjacent nodes. Self-loop make them robust to divergence of score. And normalize them as we'd learned at class time like this.

$$\hat{A} = D^{-1/2}(A + I)D^{-1/2}$$

Resulting *A hat* has sufficient information among the node relationships, also scaled with fairness. Next step is to define the LabelGCN model. I use multi-head GCN to make an ensemble model of normal GCN layer. In the code you can see *MultiHeadGCNLayer* class composed of three different heads. Each head processed input matrix as follows:

$$\text{out}_{b,j} = \frac{1}{H} \sum_{i=1}^{H} \sum_{k} \hat{A}_{jk} w_k^{(i)} X_{b,k}$$

First they multiply their internal weight with the adjacent matrix, and then multiply with input matrix. Take average their values as the output of that layer. Next we should focus on *LabelGCN* class which defines entire GCN model. After going through the dropout for preventing overfitting there are two multi-head GCN layers that the ReLU layer is caught in the middle for activation function. That two layers are sufficient for making label decision but it's better to devise a measure for oversmoothing. So I added skip-connection before returning final result by adding the very input to weigh between original and calculated value.

For training there are three important points: BCEWithLogitsLoss, AdamW optimizer, and CosineAnnealingLR scheduler. This task is multi-label classification therefore using BCE rather than softmax is useful for calculating the loss. And this function contains taking logarithm with sigmoid so that the value is quite stable. And for flexible step-size on gradient descent I used Adam optimizer with weight decay of learning rate and decay rate both equal to 5e-4 to control it and prohibiting overfitting. Not just limited to the optimizer, but also scheduler can make proper learning rate change. With CosineAnnealingLR the learning rate decreases smoothly with the cosine curve to fine-tune the update. With these three we can backpropagation process more confidently. For model is quite deep I set the initial learning /decay rate to 5e-4. You can see in every epochs the loss keep decreasing.

2.3 Model and Prediction Method

We can predict the final result by keep using our model. Within evaluation multiply the test embedding with the adjacency matrix, and take the sigmoid to make it as a probability of each class. As instructed I make restriction for the final class prediction between at least 2 and at most 3. Then provide the result as a required csv(comma seperated value) file. In conclusion we made multi-label classification prediction with Multi-head LabelGCN model using pretrained *mxbai* model.

3 Experimental results

Table 1. Comparisons among various techniques

Model	F1 score	Remarks
naive model	0.00162	TfidfVectorizer
BERT	0.01621	bert-base-uncased
BGE	0.24551	BAAI/bge-small-en-v1.5
Multi-head	0.30613	BAAI/bge-base-en-v1.5 w/ Multi-head GCN
Mixbread	0.38214	mixedbread-ai/mxbai-embed-large-v1
Self-training	0.28907	10 self-iterations

This is the table of my model improvements with the score. My initial naive model with TF-IDF method was terribly bad. So I changed the pre-trained embedding model to BERT and there was

some improvement, numerically 10 times. A remarkable score emerged when I used BGE model from BAAI(Beijing Academy of Artificial Intelligence). And when I adjust multi-head to my two layers of LableGCN model with skip-connection, finally I can reach 0.3. I noticed that more powerful embedding model led to better score. I can achieve my best score when using the embedding model from Mixedbread. But unfortunately the self-training method with 10 iterations did not make an improvement from my best score. It can be explained by negative transfer, especially confirmation bias. Silver label can be wrong, but the model trusted too much to make more proper decision.

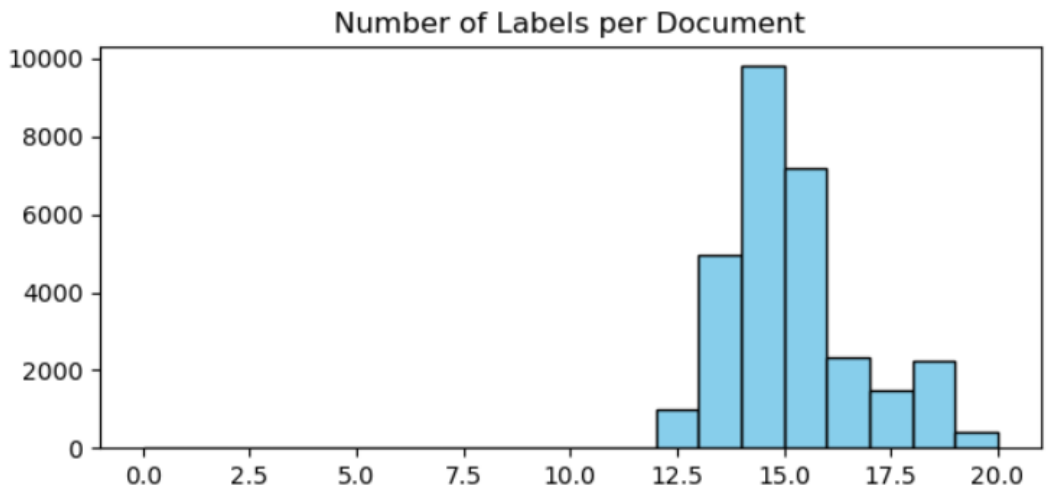

Fig. 1. Average labels per doc: 15.19

Also for pseudo-labeling there are about 15 labels per each document within the maximum restriction of 20. It look sufficient for training the GCN model with that much of silvers.

4 Case study and discussion

Let's see for good classification case. The 241st review said

"pure organic raw fruit nut bars (pack of 12) maybe if i was stuck on a desert island and only had these to eat ... then i would eat it again . i did not like the taste or the consistency"
. and my model classified this to label number 271(snack_food), 456(granola_trail_mix_bars), and 498(fruit). You can see that this prediction is reasonable because this fruit nut bar would be made of granola and fruit. Also maybe for the word 'a desert island', my model can predict this 'island' as same as 'trail trip' therefore the class 456 might be emerged.

But for the bad case, the 605th review said

"white mountain puzzles football history the way the borders work , you can distinctly tell which pieces go to which side so it really made putting the perimeter together a breeze . the main football in the center was also pretty easy to spot so it gave you a good place to work off of early . overall , we really enjoyed this puzzle"
. It can be confused but you can check this one is about the jigsaw puzzle, which is class number 6. Unfortunately the model predicted as 8(beverages), 456(granola_trail_mix_bars), and 514(sports_drinks). None of them is correct. We can check why it missed so easily. the word 'white mountain' made 8 and 456, and 'football' made 514. We can see that the word's count can be misguide the model to

predict wrong class. In this case the model should focus on the main word by reading the context carefully.

My model has a few more limitations. First of all I did not use validation nor cross-validation for early-stopping or something else. Validation data can be split from the train data very easily but lack of time I couldn't. And I did not use any LLM-call which can elaborate more about the keywords. The performance can be upgraded if I finish them all.

Received 20 December 2025